1 Feature Extractions

1A)
i) ICUType represents the type of care they are for in the ICU. It ranges from 1 to 4, 1 being Coronary care unit, 2 is cardiac surgery recovery unit, 3 medical ICU, and 4 surgical ICU. Because all values are distinct (nominal attributes) and don't have distinctive meaning/value behind the number picked, it means that a linear classifier cannot properly classify the values (especially because it is linear within a 2-dimensional plane, not 4.)
The variables can be represented as four individual variables/features with one-hot encoding. (ie. a feature will be called 'coronary care unit' and have a binary value of 1 if it is and 0 if not that ICUType.)
ii) Limitations include any variables that don't contain numerical variables (for example the ICUType mentioned prior), and potential noise/error (since it takes all values, the algorithm can't recognize when there are widely incorrect value(s).) Because this only considers the mean, it ignores important summary statistics such as count (of values), range, max & min (could be helpful to see if heart rate for example spiked or slowed for a brief moment), and threshold (how many numbers go past a certain value. ie HR >90) just to name a few.

1B)
i) This approach assumes that everyone has similar values (regardless of individuals, and other factoring data.) E.g. disregards age, weight, ICUtype, etc when assigning values.
Missing data can either be completely removed (remove full tuple), utilize some model that incorporates other values to get a better estimate for the missing value, or add weight to values that are properly present.

1C)
i) Scaling is useful to prevent unequal weights on certain variables. I.e. base heart rate could be around 60-100 range however creatinine levels should generally be around 0.5-1.5. Despite the scale, that should not push the model to prioritize one data over the other (hence why all data is normalized so the model can train on all values fairly.)

1D)
i) Dimensionality d of the feature vector is 40.
ii) Average feature vector:

	Feature Name	Mean value
0	Age	0.653507
1	Gender	0.557000
2	Height	0.390707
3	ICUType	0.578333
4	Weight	0.236098

5	mean_ALP	0.058284
6	mean_ALT	0.015867
7	mean_AST	0.013781
8	mean_Albumin	0.445900
9	mean_BUN	0.143020
10	mean_Bilirubin	0.039455
11	mean_Cholesterol	0.368926
12	mean_Creatinine	0.082277
13	mean_DiasABP	0.555640
14	mean_FiO2	0.359233
15	mean_GCS	0.720369
16	mean_Glucose	0.207822
17	mean_HCO3	0.382337
18	mean_HCT	0.416416
19	mean_HR	0.467974
20	mean_K	0.338213
21	mean_Lactate	0.076291
22	mean_MAP	0.300090
23	mean_Mg	0.158574
24	mean_NIDiasABP	0.542114
25	mean_NIMAP	0.575976
26	mean_NISysABP	0.501373
27	mean_Na	0.515676
28	mean_PaCO2	0.322203
29	mean_PaO2	0.284249
30	mean_Platelets	0.214156
31	mean_RespRate	0.350176
32	mean_SaO2	0.937895
33	mean_SysABP	0.649529
34	mean_Temp	0.674652
35	mean_TroponinI	0.136583
36	mean_TroponinT	0.046039
37	mean_Urine	0.061263
38	mean_WBC	0.090518
39	mean_pH	0.011406

2.1 Hyperparameter Selection for L2-Regularized Logistic Regression

2.1B)
It preserves the original class distribution, which can help in imbalanced datasets. For example, assume there's a low number of 'negative' labels and the original class distribution is not preserved. There is a high likelihood that there's no negative labels in a certain fold. This can result in heavily skewed training data, since there's no base case of a negative label in that training. e.g validation metrics will be incorrect, because most rely on having actual negative labels. If there isn't a single negative label within the fold, true negatives are non-existent (no real negative label, despite the algorithm attempting to predict potential negative labels.)

2.1C)

Metric	Best C	CV Performance
Accuracy	100.0	0.8665
Precision	1.0	0.6327
Sensitivity	100.0	0.2168
Specificity	0.001	1.0
F1-Score	100.0	0.3171
AUROC	1.0	0.7982
AUPRC	10.0	0.4467

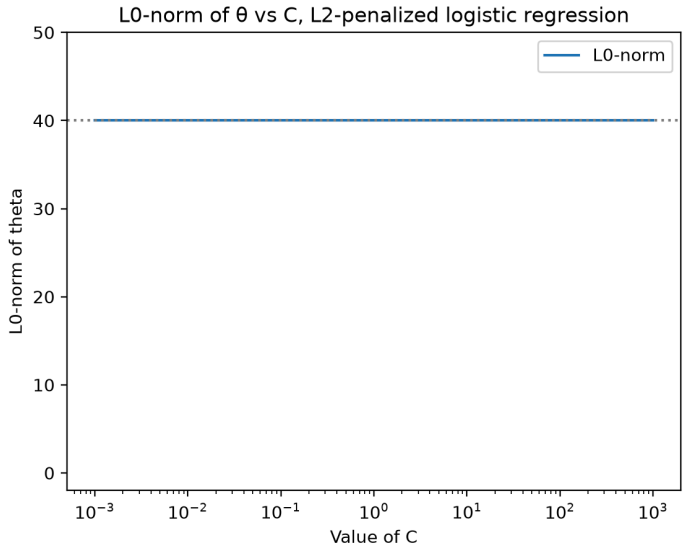
Performance measures such as accuracy, sensitivity and f1-score all prefer a C (penalty) of 100 whereas precision and AUROC perform the best on C of 1. Specificity performed perfectly on any c below 1 (values 0.1, 0.01, and 0.001, the program decided to pick 0.001 as the best despite all three C values having the same CV performance.) For the final model training I'll prioritize f1-score since it balances precision and recall (sensitivity) which provides definitive predictions during an imbalanced dataset (which this data is.) Therefore for C I would choose 100.

2.1D)

Chosen C =100	
Metric	**Test Performance**
Accuracy	0.8660
Precision	0.5758

Sensitivity	0.2639
Specificity	0.9673
F1-Score	0.3619
AUROC	0.8477
AUPRC	0.4757

2.1E)



The L0-norm of theta remains flat and seems unchanging in terms of value C because the L2 regularization shrinks every weight toward 0 (though nonzero). Therefore the count stays at 40 throughout the different penalties values, since it considers all features within the regularization.

2.1F)

Positive Coefficient	**Feature Name**
2.8777	mean_BUN
2.1420	Age
1.6587	mean_Bilirubin
1.4451	mean_Lactate

Negative Coefficient	Feature Name
-2.5321	mean_GCS
-1.6250	mean_Temp
-1.3351	mean_Na
-0.9189	Weight

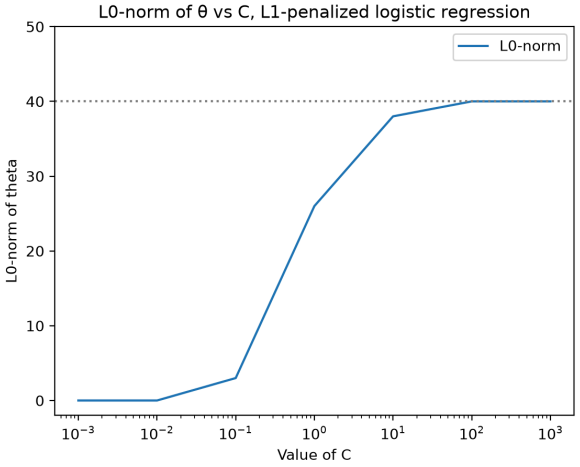
2.2 Hyperparameter Selection for L1-regularized Logistic Regression

2.2A)

Metric	Best C	CV Performance	Test Performance
Accuracy	10	0.8650	0.8600
Precision	1	0.6235	0.5455
Sensitivity	1000	0.2168	0.1667
Specificity	0.001	1.0	0.9766
F1-Score	1000	0.3153	0.2553
AUROC	1	0.7959	0.8496
AUPRC	0.001	0.5715	0.4837

(followed AUROC, so used best C of 1 for test performance)

2.2B)



The plot is very different to that of the L2-penalized logistic regression. This is because the L1 has most weights at 0. And as the penalty relaxes (increased C), more and more features are able to overcome the penalty, and the model is no longer regularized. The L1 penalty effectively determines which (and how many) features affect the model.

3 Challenge

	prediction (-)	prediction (+)
actual(-)	TN: 7123	FP: 1337
actual(+)	FN: 430	TP: 1110

f1score: 0.5568096313017307
AUROC: 0.8745890516103283

Feature engineering:
For generating the initial feature vector I made sure to split all the categorical data (gender, ICUType, and MechVent) into their own one-hot encoded variables labeled with the initial feature name followed by the category label. i.e. Gender_0 for females and Gender_1 for males. The timeseries features are sorted by time and not only valued by mean, but the full summary statistics. Impossible values are cleaned to NaN.

Hyperparameter selection:
l1_ratio=0.5, default C. l1_ratio is 0.5 to be perfectly in between L1 and L2 regularization. Noticed an underperforming AUROC so had class_weights be balanced.

Model selection:
The model I selected was elastic-net logistic regression which is a combination of both L1 and L2 regularization. This is because most of the variables/features taken can be categorized into general groups such as blood gases (lung function and acid-base balance):
PaO2, PaCO2, SaO2, pH, etc. Another category would be ‘electrolytes and minerals’ which contains Na (sodium), K (potassium), Mg (magnesium.) Elastic net is a combination of the two in which L2 helps shrink the weights of correlated features, so it will correlate groups. L1 pushes weights of unhelpful features to zero, which is the feature selection. Elastic net is therefore perfect for grouping alike features and also being able to properly select important data from the vast collection of various recorded features.

Additional Preprocessing steps:
The biggest change in the challenge code from the original was how this code handles missing values. Instead of taking the basic mean, some static variables, such as height & weight are dependent on gender, and gender is likewise dependent on available height & weight data (this is done through linear regression.) Otherwise missing values will be replaced with a general mean. The rest of the values were then dependent on linear regression of the static variables (age, gender, ICUType, weight, height.) If there wasn't enough data for the linear regression to test upon then the data will go back to the original mean of that category.
Normalization doesn't normalize based on absolute min and max and instead removes outlier data.